CMPSC 448 Midterm Project

Alexander Sabel

1. Problem Definition and Dataset Curation

As LLM-generated text becomes more and more common, it becomes important to be able to distinguish outputs between models. We can treat this as a 5-class text classification problem. Given an LLM's response and/or user input, predict which family model produced it. I investigated research questions to support this topic, such as identifying the LLM based on its response, and did the prompt in conjunction with output help a model to identify the LLM that generated it.

For this project, I utilized the LMSYS-Chat-1M public dataset from Hugging Face as my data source. It is a collection of one million conversations with multiple different LLMs, useful for the purpose of training models. From Hugging Face, this is the description of the dataset:

> *This dataset contains one million real-world conversations with 25 state-of-the-art LLMs. It is collected from 210K unique IP addresses in the wild on the Vicuna demo and Chatbot Arena website from April to August 2023. Each sample includes a conversation ID, model name, conversation text in OpenAI API JSON format, detected language tag, and OpenAI moderation API tag.*
>
> *User consent is obtained through the "Terms of use" section on the data collection website. To ensure the safe release of data, we have made our best efforts to remove all conversations that contain personally identifiable information (PII). In addition, we have included the OpenAI moderation API output for each message. However, we have chosen to keep unsafe conversations so that researchers can study the safety-related questions associated with LLM usage in real-world scenarios as well as the OpenAI moderation process. We did not run decontamination on this dataset, so it may contain test questions from popular benchmarks.*

This raw dataset had multiple different models sampled, here we can see every single different LLM:

> *[('vicuna-13b', 68321), ('koala-13b', 15793), ('alpaca-13b', 9099), ('chatglm-6b', 6047), ('fastchat-t5-3b', 5593), ('llama-13b', 5585), ('oasst-pythia-12b', 5276), ('llama-2-13b-chat', 4359), ('vicuna-33b', 3477), ('dolly-v2-12b', 3145), ('wizardlm-13b', 2847), ('vicuna-7b', 2704), ('RWKV-4-Raven-14B', 2431), ('mpt-7b-chat', 2130), ('stablelm-tuned-alpha-7b', 2086), ('claude-1', 1983), ('guanaco-33b', 1913), ('gpt-3.5-turbo', 1292), ('mpt-30b-chat', 1132), ('gpt4all-13b-snoozy', 1074), ('gpt-4', 1052), ('palm-2', 906), ('claude-instant-1', 802), ('llama-2-7b-chat', 623), ('claude-2', 330)]*

I then had to curate this dataset into conversations that could be trained on. I selected conversations that were English, single-turn (one prompt and one reply), and not redacted. Each conversation became a (model, input, output) row. I then grouped raw model names into five families by keyword (claude, gpt, llama, mpt, vicuna) and selected a sample of 3,000

conversations from each. Finally, I removed conversations with very short inputs or outputs, and removed duplicates. Here were the conversation totals for each group:

family	
vicuna	*2731*
gpt	*2707*
claude	*2685*
llama	*2677*
mpt	*2675*

2. CNN/RNN Implementation and Training

I split conversations 70/10/20 into train/validation/test sets. Text is lowercase and tokenized with regular expression into words, single punctuation characters, and newline tokens. This was done because other identifying text features can act as fingerprints for different LLM models. Vocabulary was built from the training split only, giving 24,410 entries. Sequences are shortened or padded to 256 tokens for outputs and 64 for prompts.

CNN

Embedding (128-d) → three parallel 1D convolutions (kernel sizes 3, 4, 5; 100 filters each) with ReLU → max-over-time pooling → concatenation (300-d) → dropout (0.5) → linear layer (5 classes).

RNN

Embedding (128-d) → bidirectional LSTM (hidden size 128 per direction) → max-pooling over non-padding time steps (padding is masked out) → dropout (0.5) → linear layer (5 classes).

Training

Both models use cross-entropy loss, batch size 64, and up to 15 epochs. Early stoppage was applied to check accuracy after every epoch, and training is stopped after 3 epochs without improvement. Every configuration was trained across 3 random seeds. In total, 18 neural models (2 architectures * 3 input modes * 3 seeds) were trained. I used Google Colab, utilizing their T4 GPU to improve runtime. I used TF-IDF with logistic regression as a baseline (max 1,000 iterations).

3. Result and Analyses

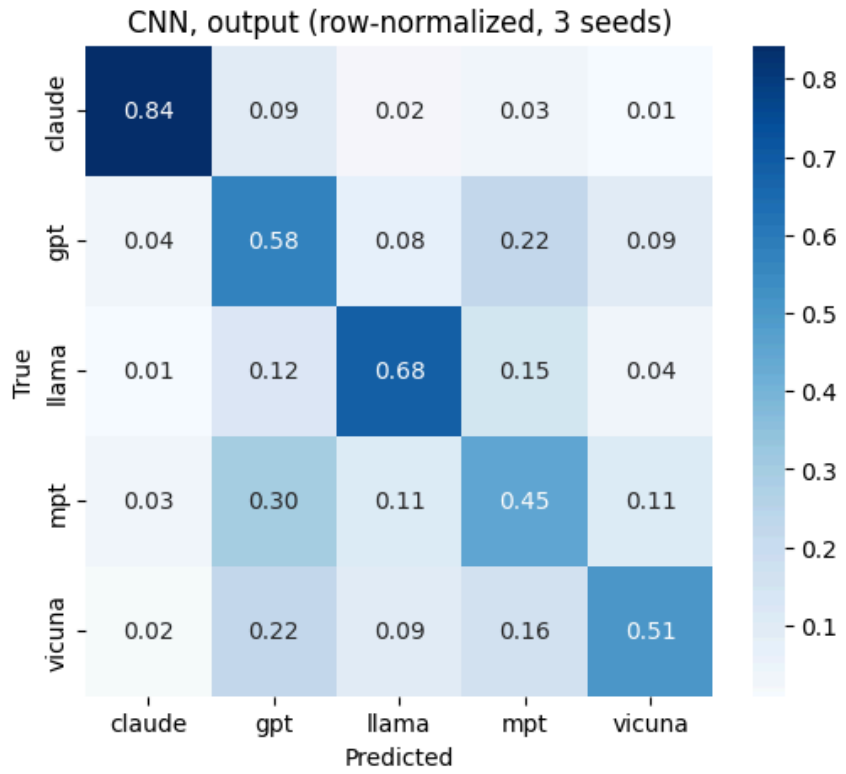
RQ1

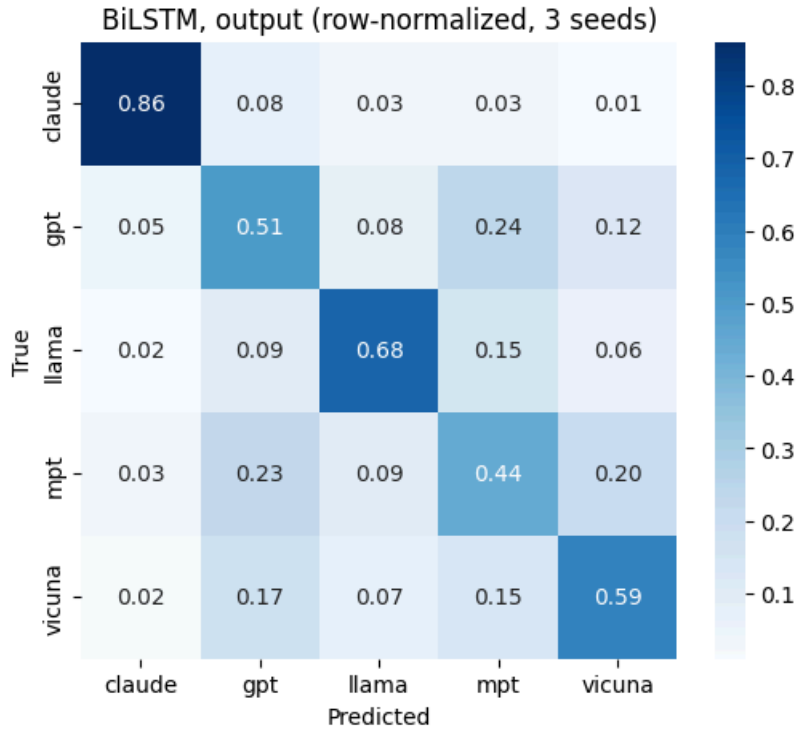
Using the model, it is possible to identify which LLM generated a response. The CNN reached 61.5% ± 0.2% test accuracy (macro-F1 0.617) and the BiLSTM reached 61.9% ± 1.0% (macro-F1 0.618) across 3 seeds, compared with 20% chance for five classes. Both beat a TF-IDF + logistic regression baseline (57.1% accuracy, 0.571 macro-F1). The CNN and BiLSTM differ by less than one seed standard deviation, so we cannot say either architecture is better.

We can also see the precision, recall, and F1 for each family across CNN and BiLSTM. We can see from the table that Claude is by far the easiest to identify with highest precision, recall, and F1 by far. Llama is the second best, followed by Vicuna, and then GPT and MPT. The same pattern can be seen across both models, so this is probably an actual legible difference in being able to identify these models.

CNN				
Family	**Precision**	**Recall**	**F1**	**Test n**
claude	0.905	0.843	0.872	1689
gpt	0.443	0.58	0.502	1602
llama	0.686	0.682	0.684	1578
mpt	0.451	0.454	0.453	1626
vicuna	0.667	0.507	0.576	1602
BiLSTM				
Family	**Precision**	**Recall**	**F1**	**Test n**
claude	0.887	0.86	0.873	1689
gpt	0.469	0.507	0.487	1602
llama	0.71	0.68	0.694	1578
mpt	0.444	0.445	0.444	1626
vicuna	0.599	0.594	0.596	1602

Row-normalized confusion matrix for the CNN and BiLSTM on output-only text, summed over 3 seeds. We can see from these graphs what model tends to get confused with which model. At the higher end, the models tend to do well with identifying Claude, and on the lower end mpt and gpt tend to get mixed up between the two as opposed to any other models.


CNN, output (row-normalized, 3 seeds)
True
Predicted
claude
gpt
llama
mpt
vicuna
0.84 0.09 0.02 0.03 0.01
0.04 0.58 0.08 0.22 0.09
0.01 0.12 0.68 0.15 0.04
0.03 0.30 0.11 0.45 0.11
0.02 0.22 0.09 0.16 0.51
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1


BiLSTM, output (row-normalized, 3 seeds)
True
Predicted
claude
gpt
llama
mpt
vicuna
0.86 0.08 0.03 0.03 0.01
0.05 0.51 0.08 0.24 0.12
0.02 0.09 0.68 0.15 0.06
0.03 0.23 0.09 0.44 0.20
0.02 0.17 0.07 0.15 0.59
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1

RQ2

I compared three input settings for both the CNN and BiLSTM: prompt only, response only, and prompt+response. Each configuration was trained with 3 random seeds, and the table below reports test accuracy and macro-F1 with standard deviation. From the table, we can see that about 1.5 points of accuracy are gained across both models when adding the prompt. Because the prompt contains no model written text, this small increase in accuracy may just be from the differences in how the models can receive text, and not in how they process anything.

TEST ACCURACY (mean ± std over 3 seeds)

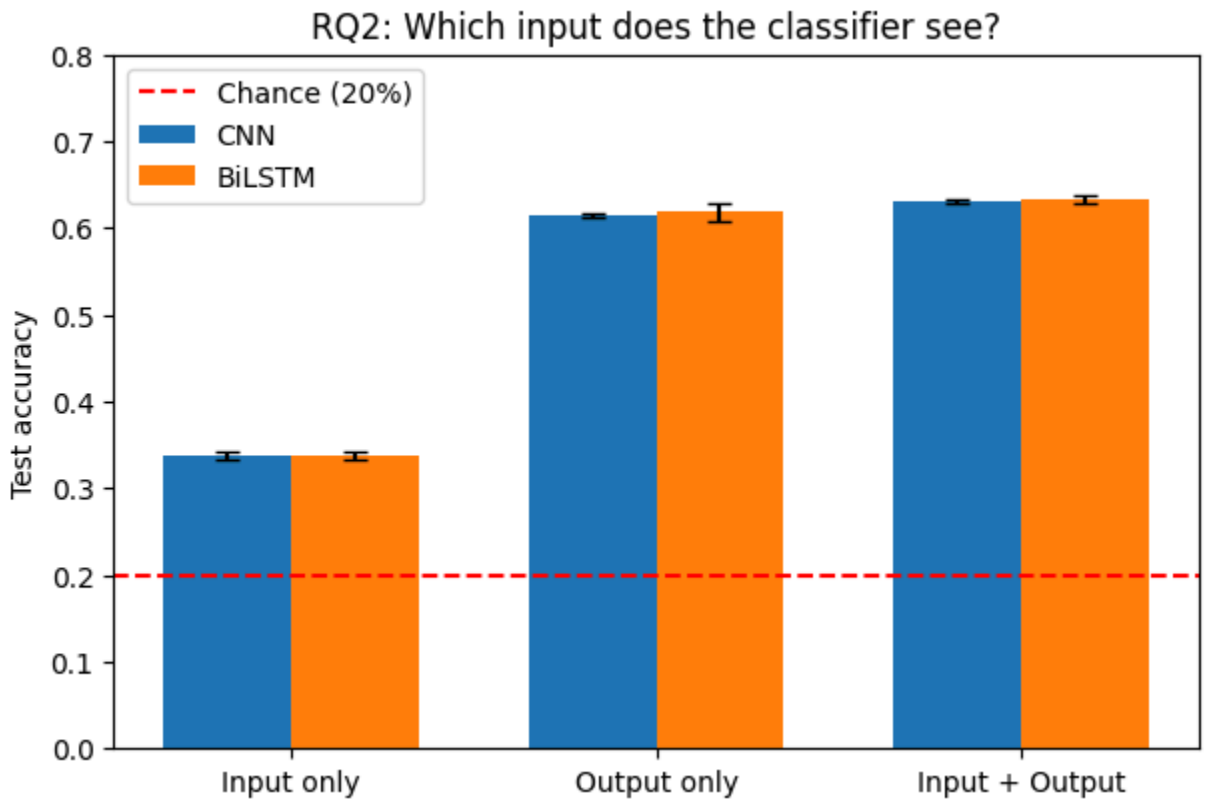
	Input only	Output only	Input + Output
CNN	0.339 ± 0.005	0.615 ± 0.002	0.631 ± 0.002
BiLSTM	0.337 ± 0.005	0.619 ± 0.010	0.633 ± 0.004

TEST MACRO-F1 (mean ± std over 3 seeds)

	Input only	Output only	Input + Output
CNN	0.340 ± 0.005	0.617 ± 0.001	0.634 ± 0.002
BiLSTM	0.337 ± 0.006	0.618 ± 0.012	0.636 ± 0.002

Chance level: 0.2

Here we can see a bar chart of how the differences across input, output, or input+output effect model accuracy. Visually there is a small up-tick in accuracy, but by a very miniature margin:



Using only the user's prompt, the CNN reached 33.9% ± 0.5% test accuracy and the BiLSTM 33.7% ± 0.5% (input-only TF-IDF + logistic regression reached 35.6%), about 14 points above the 20% chance level for five classes but far below the roughly 62% achieved from the response alone, so the prompt carries some information about which model answered, though much less than the response does. Adding the prompt to the response raised accuracy only slightly, from 61.5% to 63.1% for the CNN (+1.6 points) and from 61.9% to 63.3% for the BiLSTM (+1.4 points), a gain that is consistent across both models and both metrics. It is clearest for the CNN, whose seed variation is small (± 0.2 points), but only slightly larger than seed variation for the BiLSTM (± 1.0 points), so we treat it as suggestive rather than conclusive. The above-chance input-only result most likely reflects differences in which prompts each model received rather than a stylistic fingerprint, since the prompt contains no model-written text and would sit at chance if every model had answered the same prompts. Our prompt analysis is consistent with this: median prompt length was shorter for Claude and GPT (13 words) than for Llama, MPT, and Vicuna (19 to 20 words), Vicuna prompts contained code-related keywords about twice as often (20.3%) as the other families (9.2% to 11.8%), and the most discriminative prompt phrases pointed to templated task types (for example, "single choice question" for Llama and "2000 words" for Vicuna), while non-ASCII rates were nearly identical across families (0.1% to 0.2%), so language differences do not explain the effect. We did not verify how LMSYS-Chat-1M assigned prompts to models, so this explanation is likely rather than proven. Because our split groups only by exact prompt text, near-duplicate templated prompts may also appear in both train and test and could inflate input-only accuracy somewhat. A cleaner design would send identical prompts to every model, which should push input-only accuracy toward chance, and the small gain from adding the prompt to the response may partly come from the same effect.

RQ3

Performance fell on unseen task types, but by very different amounts in the two directions. A CNN trained on non-code prompts scored 60.2% on non-code test data and 54.2% on code data, a drop of 6 points. A CNN trained on code prompts scored 60.1% in-domain but only 39.0% on non-code data, a drop of 21 points. Both unseen-task scores remain well above the 20% chance level, so part of what the classifier learns carries across tasks, but the large drop when training on code suggests it also relies on task-specific patterns. The asymmetry is likely partly a data-size effect, since the code subset is much smaller (see Table X for sizes), so a model trained on it sees less varied text. The task labels come from a crude keyword rule, so these results are approximate.

CNN, Output only

Trained on	Tested on same task	Tested on unseen task	Drop
other	0.602	0.542 (code)	-6.0 points
code	0.601	0.390 (other)	-21.1 points

Removing formatting and punctuation signals lowered accuracy, but never close to chance. Stripping markdown symbols and newlines cost the most (61.5% to 57.1%), followed by truncating responses to 40 words (58.3%) and removing punctuation (59.0%). The seed standard deviations are 1.2 points or less, so these drops are larger than seed noise. Formatting and structure therefore contribute to identification, but the models retain most of their accuracy without them, which suggests the main signal is in the word choice and phrasing rather than any single structural feature. Truncation to 40 words still leaves accuracy at 58.3%, so identifying cues appear early in a response.

Condition	Test accuracy	Change
Original	0.615	n/a
Strip markdown and newlines	0.571 ± 0.012	-4.4 points
No punctuation	0.590 ± 0.002	-2.5 points
First 40 words only	0.583 ± 0.004	-3.2 points

4. Lessons Learned and Experience

I learned a couple good lessons during this assignment. It was interesting how seeds and baseline give important context and comparisons, and can be used to eliminate the issue of noise. Running three seeds showed that the difference between the CNN and BiLSTM was within seed variation, so without them I might have wrongly claimed one architecture was better. The baseline showed that the neural models add only a few points more than a simple linear model. This was a good reminder that a more complex model is not automatically a better one, and that a simple baseline is worth running first. I also think a limiting factor of this work was similar models, for example Vicuna is a branched model of Llama, so getting an accuracy ~60% can be pretty good considering these factors. Another limiting factor was that each family combines several model versions, such as GPT-3.5 and GPT-4 in one class, and the GPT family may also include gpt4all, which is not an OpenAI model. The confusion matrices showed that GPT and MPT were the hardest to separate, while Claude was identified far more easily than the others, which I did not expect going in. Vicuna was also often labeled as GPT, which may be because it was fine-tuned on ChatGPT-style conversations, though I did not test this. Implementing early stoppage was also useful for saving processing time, and although might not be a big deal here with 15 total epochs, with a larger dataset and more epochs can be much more crucial. It also kept the models from overfitting, since the best-epoch weights were restored before testing. There were some limitations added due to the preprocessing, such as reducing conversations to 256 tokens. Lowercasing the text also removed capitalization, which could have been a useful stylistic signal. The RQ2 results taught me that the dataset itself can

leak information: the prompt alone predicted the model at about 34%, well above the 20% chance level, even though it contains no model-written text, which suggests that different models received different kinds of prompts. This showed me that how data is collected can matter as much as the model, and that a stronger design would send the same prompts to every model. Splitting by prompt and building the vocabulary from the training data only were also important for keeping the test results honest, although near-duplicate prompts could still appear in both train and test. In the future being able to access a larger dataset, with more and newer models, may help improve the accuracy of the model. It would also help to have every model answer the same prompts, to test pretrained transformer models, and to use more than three seeds so that small differences can be judged with more confidence.